

# Meridian Project

## Problem Context

You have just been hired at **Meridian**, a company of 200 employees working in a hybrid setup.

Your first day starts in **3 days**.

You don't know anyone.

You don't know how things work.

The only email you received from HR says:

"Welcome! See you on Monday."

As the newest employee, **you are the first developer trying to solve this onboarding problem.**

Your task is to build a **web application** that makes the first month at Meridian significantly less chaotic during the onboarding process.

Build **the application you wish you had on your first day at work.**

## What You Know About Meridian

- Hybrid work model: **3 days in the office, 2 days remote**
- Departments:
  - Engineering
  - Sales
  - Marketing
  - HR
  - Finance
- Internal communication tools:
  - Slack
  - Google Meet
- The company hires **2–3 new employees per month**
- HR consists of **a single person**

Everything else about how the company operates is up to you.

## Requirements

### Recommended Technology Stack

We recommend using:

- .NET
- React / Angular (TypeScript-based)
- or Python
- Any database of your choice

### Authentication

Authentication is **optional**.

If you decide to include it, it should make sense in the context of your application.

### Deployment

Deployment is **not required**.

However, your repository **must include a README** with clear instructions explaining how to run the application locally.

The following decisions are entirely yours:

- Application architecture
- Libraries and frameworks
- Database structure
- Data model
- User workflows
- UI/UX decisions

# Required Documentation

## ASSUMPTIONS.md

Describe what assumptions you made and why.

Suggested structure:

### About the users

- Who uses the application? (new employee, HR, managers, colleagues...)
- What does the user already know when opening the application for the first time?

### About the data

- Who enters the information into the application?
- When is the information added?
- What happens if information is missing or incorrect?

### About the context

- What device does the new employee use on the first day?
- Do they have access to the application before their first working day?

## DECISIONS.md

Explain what you decided to build, what you intentionally did **not** build, and why.

Suggested structure:

### Product decisions

- Which features did you include?
- How did you prioritize them?
- Which features did you intentionally leave out of scope?

### Technical decisions

- Why did you choose this database structure?
- Why did you choose these libraries/frameworks?
- If you had more time, what would you build differently?

### UX decisions

- Why did you choose this user flow?
- Did you test it with anyone?

- What changed after receiving feedback?

## WHAT\_I\_WOULD\_DO\_NEXT.md

Imagine you had **two additional weeks**. What would you build next?

Priority 1 — Features that would fundamentally improve the experience

Priority 2 — Features that would add significant value

Priority 3 — Nice-to-have improvements and why they matter

## REFLECTION.md *(Optional, but strong differentiator)*

Reflect on your work.

Some questions to consider:

- What turned out to be harder than you expected?
- Which decision would you make differently if you started over?
- What did you learn about yourself as a developer during this project?

## Deliverables

### Source Code

A **public Git repository** (GitHub or GitLab).

### Demo Video

Record a walkthrough of your application demonstrating its main workflows.

Requirements:

- Maximum **3 minutes**
- Save the video using the format:  
**Firstname\_Lastname**
- Everything must be **in English**.
- Deliver it as one of the following:
  - an **Unlisted YouTube link**
  - a **Google Drive link** with access enabled for anyone with the link

- o a file stored in your Git repository (only if the file size allows it)

### A Few Things to Keep in Mind

There is **no single correct solution**.

The challenge is intentionally open-ended. How you deal with ambiguity is an important part of what we evaluate.

We will review your **Git commit history**, so work incrementally rather than making a single large commit at the end.

If you realize that one of your initial assumptions was wrong or that you changed direction during development, **document it**. We see that as a positive sign of thoughtful engineering.

Your demo video **does not need professional editing or production quality**.

We simply want to see your application working, in no more than **3 minutes**, what you built and why.